

# 비용 및 토큰 트레이싱



# 모델 품질과 시스템 성능

품질 점수는 답의 내용을, 시스템 지표는 응답의 속도와 비용을 다룬다.

## 모델 품질 · 정확도 계열

### 측정 대상

답의 내용이다. 근거를 찾았는지, 답변이 근거에 충실한지를 본다.

### 측정 방법과 시점

고정 평가셋과 채점자로 배포 전에 측정한다. 값이 하나로 요약된다.

## 시스템 성능 · 속도·비용 계열

### 측정 대상

응답의 속도와 비용이다. 요청당 걸린 시간, 단위 시간당 처리량, 요청당 비용을 본다.

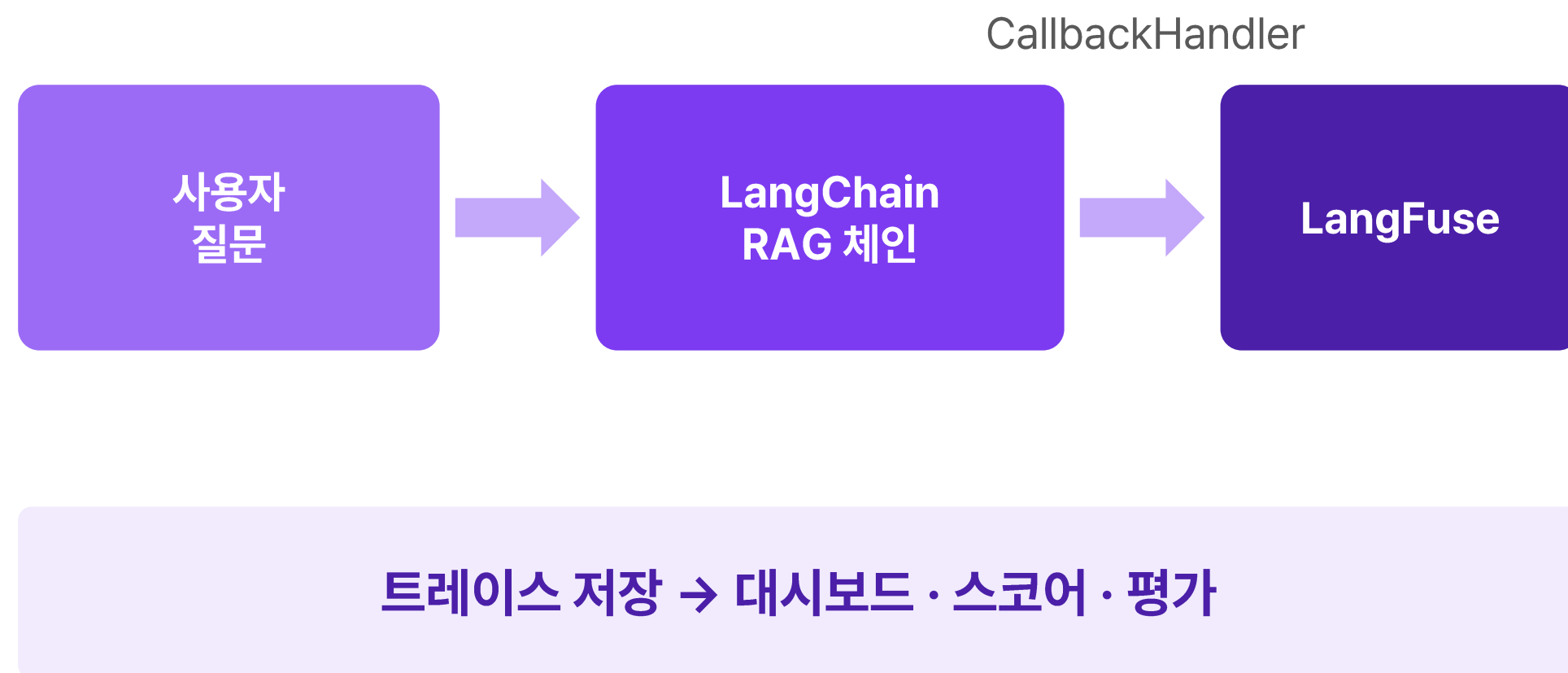
### 측정 방법과 시점

실행 기록과 시계열로 배포 후에도 계속 측정한다. 분포로 관리한다.



나빠질 때의 증상도 다르다. 정확도가 내려가면 틀린 답이 늘고, 시스템 지표가 나빠지면 사용자가 기다리다 이탈한다.

# Observability Tool: LangFuse 예시



- Observability Tool은 요청마다 프롬프트·응답·토큰·지연·비용과 중간 단계(검색·생성)를 구조화된 트레이스로 기록하는 도구이다.
- LangFuse는 그 예시이다. 오픈소스이며, 기록 위에서 대시보드·스코어·LLM-as-a-Judge 평가·프롬프트 관리로 이어진다.
- 실습에서는 같은 기록을 `traced_call`로 직접 남기고 운영에서는 이런 도구가 수집을 대신한다.



## Trace 안에 단계(Observation)가 중첩된다.

RAG 한 번의 질문이 하나의 Trace가 되고, 그 안에 retrieval·generation 단계가 중첩된다

Langfuse v3.82.0

Langfuse Demo / langfuse-docs / Traces

Go to...

Home

Dashboards

Observability

Tracing

Sessions

Users

Prompt Management

Prompts

Playground

Evaluation

Scores

LLM-as-a-Judge

Human Annotation

Datasets

Settings

Support

FK Felix Krauth felix@langfuse.com

Trace: eb346ba8-19d9-4368-9b71-022cd17f25d7 with-context ☆ 🔗

Search (type, title, id) Timeline

qa 11.58s \$0.000371

retrieval 1.55s \$0.000001

prompt-embedding 0.67s 12 → 0 (\$12) \$0.000001

vector-store 0.69s

context-encoding

fetch-prompt-from-langfuse 0.47s

generation 9.57s 1301 → 291 (\$1592) \$0.00037

qa ID

2025-07-16 23:44:16.449

Session: lf.docs.conversation.s7A0W7i User ID: u-M9QTUzj Env: default Latency: 11.58s Total Cost: \$0.000371

1,313 → 291 (\$1,604) Release: 044d0162fe3dbddd09f88a554521ddf73726618c

Preview Scores Formatted JSON

Input View as markdown

"How to update the input and output in langgraph traces?"

Output View as markdown

"To update the input and output in Langfuse traces, it's essential to understand that Langfuse allows meticulous tracking of steps within the AI process by logging both inputs and outputs of large language models (LLMs). This means that every interaction you have can be captured and displayed transparently within the UI.  
  
Here's how you can effectively manage and visualize these updates:  
  
### Steps to Update Traces  
  
1. \*\*Log Inputs and Outputs\*\*: Ensure you're logging the inputs and outputs of  
  
...expand (989 more characters)  
"

Metadata

{  
 "pathname": "/docs/demo"  
}

1

### 실패 재현

단계마다 입력·출력·소요 시간이 그대로 기록되어 실패를 재현할 수 있다

2

### 비용 집계

토큰 수와 비용, 전체 지연(Latency)이 요청 단위로 집계된다

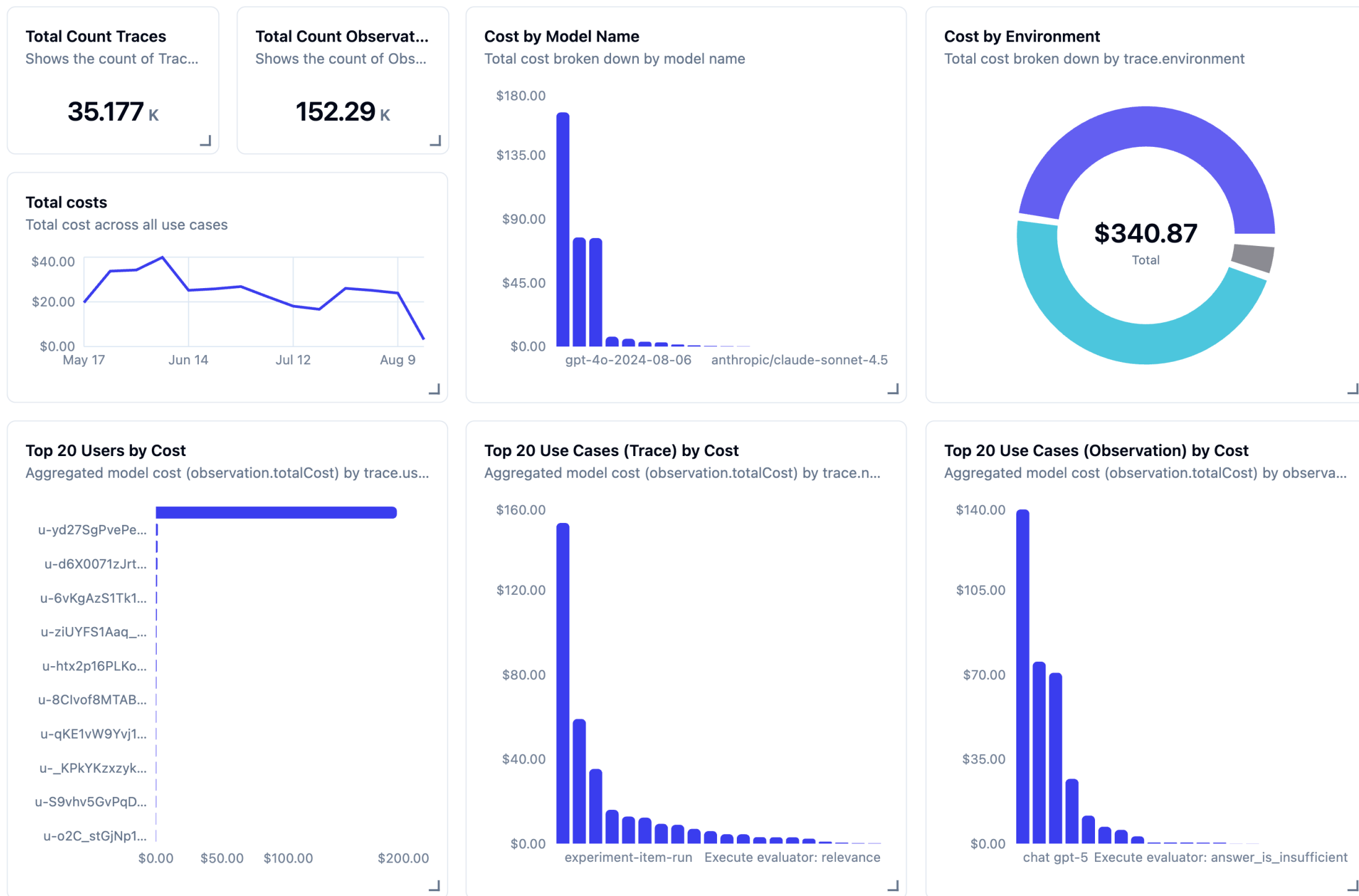
3

### 단위 확장

Session·User ID로 묶으면 대화·사용자 단위 분석으로 확장된다

출처: LangFuse 공식문서 (langfuse.com/docs)

langfuse.openai 래퍼로 호출을 바꾸면 usage가 자동 수집·집계된다.



## 요청·사용자·모델별 비용 자동 집계

**1 비용 분해**  
모델·사용자·기간별로 비용을 나누어 확인하며 이상 급증을 찾는다

**2 역추적**  
트레이스마다 토큰·비용이 기록되어 비용이 큰 요청을 역추적할 수 있다

여기에 집계된 지표가 릴리즈 게이트에서  
비용 조건을 판정하는 근거가 된다.



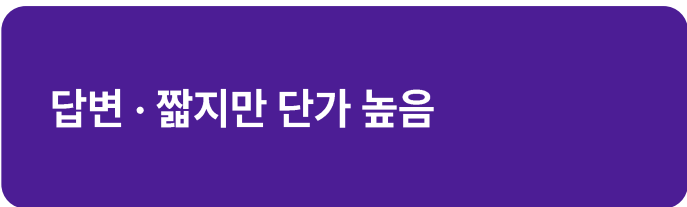
# 요청 1건의 토큰 구조

요청 1건의 비용은 토큰 구조로 정해진다. 입력의 대부분이 검색 컨텍스트이다.

## 입력 토큰 (Input)



## 출력 토큰 (Output)



### 비용 계산

입력 단가 × 입력 토큰  
+ 출력 단가 × 출력 토큰

단가는 1M 토큰 기준  
청구 기준은 usage 실측

절감은 컨텍스트에서 시작한다

### Top-K 축소

5 → 30이면 입력이 바로 줄어든다  
검증: Recall 재측정 · 실습 2

### 청크 길이·프롬프트 정리

같은 K에서도 토큰이 줄어든다  
검증: Faithfulness 전후 비교 · 실습 7

### 캐싱·모델 선택

반복 입력 할인 · 단가 자체 인하  
검증: RAGAS A/B 비교 · 실습 8

같은 내용을 담은 한국어 텍스트는 영어보다 토큰이 약 1.3~1.5배 더 필요하다(최신 토크나이저 기준). 비율은 모델마다 다르므로 실제 사용 모델로 측정한다.

수단마다 확인해야 하는 지표가 다르므로, 절감안과 검증 지표를 한 쌍으로 묶어 둔다.



# API 가격 구조

| 모델 (예시)                | 입력 \$/1M | 출력 \$/1M | 출력/입력 비율 | 용도             |
|------------------------|----------|----------|----------|----------------|
| gpt-5.4                | 1.25     | 10.00    | 8배       | 본 과정 표준 모델     |
| gpt-5.4-mini           | 0.15     | 0.60     | 4배       | 경량 작업·비용 절감 후보 |
| text-embedding-3-small | 0.02     | -        | -        | 임베딩 (출력 과금 없음) |
| 캐시된 입력                 | 할인 적용    | -        | -        | 반복 프롬프트 절감     |

단가는 구조 이해를 위한 예시 값이다. 실제 단가는 시점·벤더마다 다르므로 청구서와 공식 가격표로 확인한다.



# 캐싱·배치로 단가를 낮추는 방법

## WAY 01

### 프롬프트 캐싱

- 반복되는 앞부분 입력을 재사용한다. 시스템 프롬프트와 예시가 고정일 때 효과가 있다.

## WAY 02

### 배치 처리

- 즉시 응답이 필요 없는 작업을 모아 보낸다. 평가 실행이나 색인 갱신이 대상이다.

## WAY 03

### 적용 조건 확인

- 캐싱은 앞부분이 실제로 같아야 하고, 배치는 지연을 감수할 수 있어야 한다.

절감 여부는 청구 기준 사용량으로 확인한다. 캐시 적중 비율과 배치 처리 건수를 함께 기록하면 실제 절감 폭이 드러난다.



# usage 기반 실측 비용 계산

## 비용 계산

```
PRICES = {
    "gpt-5.4": {"in": 1.25, "out": 10.00},
    "gpt-5.4-mini": {"in": 0.15, "out": 0.60},
}

def cost(model, in_tok, out_tok=0):
    p = PRICES[model]
    return (in_tok / 1e6 * p["in"]
            + out_tok / 1e6 * p["out"])

resp = client.chat.completions.create(...)
usage = resp.usage
c = cost("gpt-5.4", usage.prompt_tokens,
         usage.completion_tokens)
```

- 추정치와 실측(usage)은 다를 수 있다.  
청구 기준은 usage이다.
- usage 입출력 토큰에 단가를 곱한 값이 요청당 비용이다.
- 요청당 비용 × 월 트래픽으로 월 예산 규모가 나온다.

비용은 감각이 아니라 usage 실측으로 측정한다. 집계는 실행 기록에서 자동화할 수 있다.



# 비용 절감 전략 세 가지

비용은 세 지점에서 줄일 수 있다. 전략마다 절감 폭과 품질 위험이 다르다.

## 1 컨텍스트 축소

Top-K와 청크 길이를 줄여  
입력 토큰을 줄인다.  
절감 폭이 가장 크고 빠르다.

절감 지점

입력 토큰

위험 요인

Recall 하락 위험이 있다.

## 2 프롬프트 캐싱

시스템 프롬프트를 정리하고  
반복 입력은 캐싱한다.

절감 지점

반복 입력

위험 요인

오래된 캐시 응답이 위험 요인이다.

## 3 모델 선택

경량 모델로 바꾸면  
단가 자체가 내려간다.

절감 지점

모델 단가

위험 요인

품질 전반이 흔들릴 수 있어  
검증 부담이 가장 크다.

절감 폭이 가장 큰 수단과 검증 부담이 가장 큰 수단이 다르다. 어느 것을 먼저 시도할지는 이 대비를 보고 정한다.



# 절감안의 품질 방어 절차

절감에는 품질 저하가 따를 수 있다. 순서를 정해 두면 되돌릴 지점이 분명해진다.

## 1단계 · 절감 적용

한 번에 한 가지만 바꾼다

## 2단계 · 지표 재측정

### 같은 평가셋으로 다시 측정

검색 지표와 생성 지표를 함께 본다

## 3단계 · 게이트 판정

품질·지연·비용 세 조건으로

통과 여부를 정한다

## 절감률에 따른 검증 강도

절감이 작으면 검색 지표 확인으로 충분한 경우가 있고, 절감이 크면 생성 지표와 사람 표본 채점까지 함께 본다



## 미달이면 원래 구성으로 복귀

차단 사유를 백로그에 기록

여러 절감안을 한꺼번에 적용하면 어느 것이 품질을 떨어뜨렸는지 알 수 없다. 하나씩 적용해 측정하는 것을 권장한다.

# Latency 및 QPS 모니터링



# 지연과 처리량의 기본 개념

## Latency · 지연

요청 하나가 시작부터 끝까지 걸린 시간이다.  
요청마다 값이 달라 한 값으로 요약되지 않는다.

## TTFT

첫 토큰이 도착할 때까지의 시간이다. 사용자가  
화면에서 기다린다고 느끼는 구간이다.

## p50 · p95

지연 분포의 중간값과 상위 5% 경계이다. 평균  
대신 이 두 값으로 관리한다.

## QPS · 처리량

초당 처리한 요청 수이다. 지연이 한 요청의  
시간이라면 처리량은 단위 시간당 건수이다.

## 동시성

같은 시각에 처리 중인 요청 수이다. 동시성이  
오르면 대기열이 생겨 지연이 늘어난다.

정확도 지표는 평가셋 하나당 하나의 값으로 요약되지만, 시스템 지표는 요청마다 값이 달라 분포로 관리한다.



# 지연이 서비스에 미치는 영향

지연 관리가 필요한 이유는 다음 4가지이다.

01

**첫 토큰까지의 대기가 체감의 핵심이다**

총 완료 시간이 같아도 첫 글자가 늦으면 느리다고 느낀다

02

**동시 사용자가 늘면 대기열이 생긴다**

같은 시스템이 특정 부하를 넘는 순간부터 급격히 느려진다

03

**타임아웃과 재시도로 같은 비용이 두 번 발생한다**

응답을 받지 못해도 이미 처리한 토큰의 비용은 청구된다

04

**응답 시간 상한이 요구되는 경우가 있다**

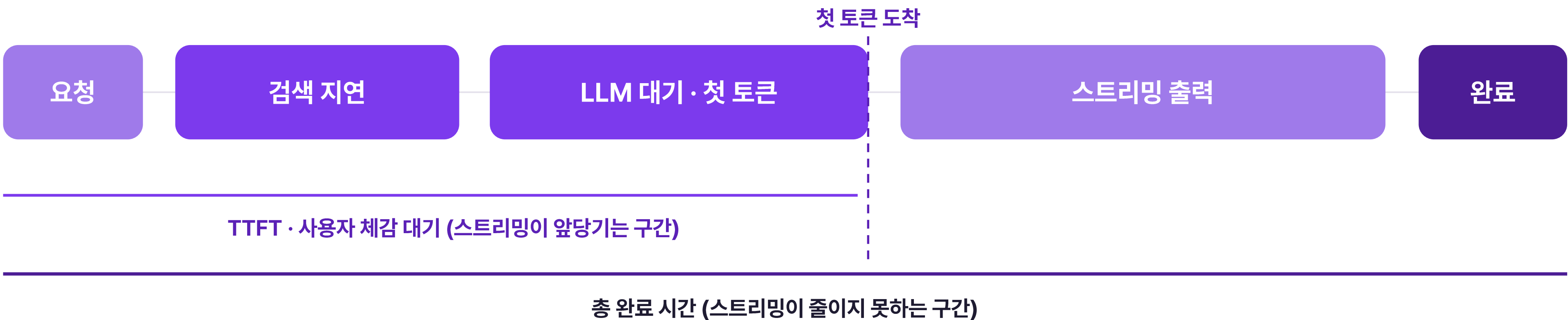
상담 보조나 사내 검색처럼 대기 한도가 정해진 화면이 있다  
이 상한을 SLO(서비스 수준 목표)로 정해 p95 기준으로 관리한다

정확도는 배포 전에 확정할 수 있지만, 이 4가지는 실제 부하가 걸린 뒤에 드러난다. 따라서 배포 후에도 계속 측정한다.

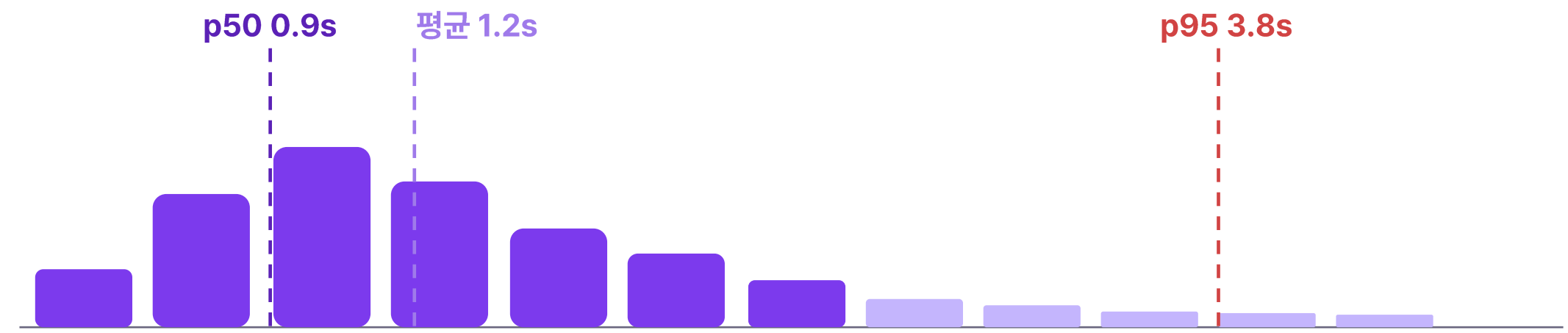


# 응답 지연의 구조

지연은 하나의 숫자로 요약되지 않는다. 첫 토큰과 완료 시점을 나누어 측정하고, 분포의 상위 구간(p95)으로 관리한다.



분포로 보는 지연 · 평균에는 느린 요청이 드러나지 않는다



## 부하에 따른 변화 · 동시성

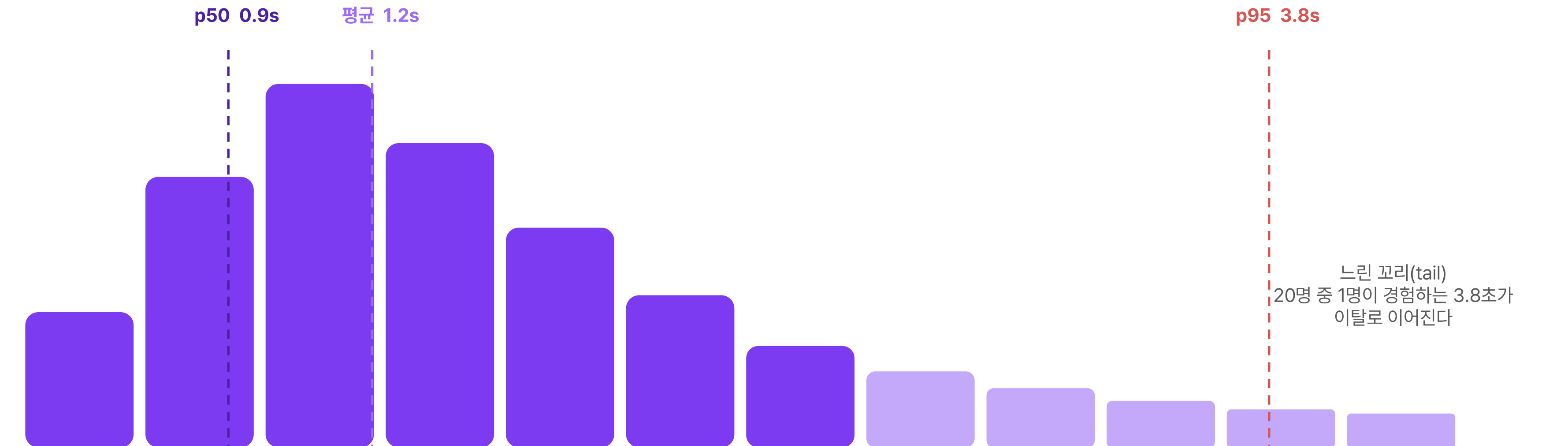
동시성이 낮은 구간에서는 지연이 서서히 증가한다  
특정 부하를 넘으면 지연이 비선형으로 급증한다  
따라서 지연 수치는 측정 당시의 부하 조건과 함께 기록한다

첫 토큰과 완료 시점, 분포, 부하 변화를 함께 보아야 지연의 전체 모습이 드러난다. 수치는 예시이다.



## 평균·p50·p95

평균에는 느린 소수의 요청이 드러나지 않는다. 분포로 보면 이탈이 어디서 생기는지 확인할 수 있다.



모니터링과 SLO는 평균이 아니라 p95(필요하면 p99) 기준으로 정한다. 수치는 예시이다.

개선 수단마다 어떤 지표가 올라가고 어떤 지표가 나빠지는지 함께 본다.

| 정확도 개선 수단 | 올라가는 지표          | 나빠지는 지표        | 관련 내용                 | 판단 기준                     |
|-----------|------------------|----------------|-----------------------|---------------------------|
| Top-K 확대  | Recall@K         | 지연 · 입력 토큰     | 「K 선택 절차」             | Recall 상승분이 지연 증가를 정당화하는가 |
| 질의 재작성    | Recall · MRR     | 지연 (LLM 호출 추가) | 「Query Rewriting의 비용」 | 재작성이 오히려 실패하는 질문이 얼마나 되는가 |
| 재순위 추가    | Precision · NDCG | 지연 (추론 단계 추가)  | 「하이브리드 검색과 재순위」       | 상위 순위 개선이 체감으로 이어지는가      |
| 큰 모델로 교체  | 답변 품질            | 지연 · 요청당 비용    | 「Judge 모델 선택」         | 품질 상승분이 비용 증가에 비해 큰가      |

정확도 개선 수단은 대체로 응답성이나 비용을 대가로 요구한다. 어느 쪽을 얼마나 양보할 수 있는지는 서비스 요구사항에 따른다.



# 지연 예산 설계

체감 한도에서 역산해 구간마다 시간을 배분한다.

지연 예산 (예: 총 3.0초)



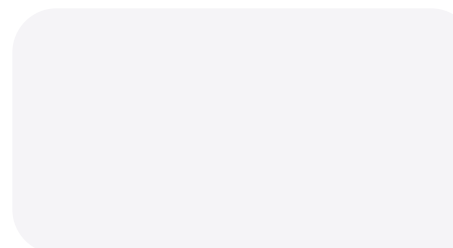
검색 0.4초



재작성 0.3초



생성 1.8초 · 첫 토큰 0.8초 + 출력 1.0초



여유 0.5초

## 예산 계산

체감 한도 3.0초에서 시작

– 검색·재작성·여유 1.2초

= 생성 구간 1.8초

상한은 p95 기준으로 둔다

## 초과 구간 판별

실측 p95가 상한을 넘는 구간부터 개선한다

## 예산을 넘는 개선안의 처리

### 구간 재배분

다른 구간에서 시간을 줄여 확보한다

판단: 총 예산 안에 들어오는가

### 예산 상향 협의

체감 한도 자체를 다시 정한다

판단: 화면 요구사항과 맞는가

### 개선안 보류

정확도 상승분이 작으면 되돌린다

판단: 게이트의 지연 조건 통과 여부

예산은 평균이 아니라 p95 기준으로 정한다. 평균에 맞추면 느린 쪽 요청이 상한을 넘는다.

처음 배분은 가설이고, 실측 p95가 배분을 조정하는 근거가 된다. 수치는 예시이다.



# 스트리밍과 체감 지연

스트리밍은 첫 토큰까지의 시간을 앞당기지만 총 완료 시간은 줄이지 않는다.

## 총 완료 시간

## TTFT · 첫 토큰까지

스트리밍  
적용

### 그대로이다

토큰을 모두 생성하는 시간은 변하지 않는다  
서버 자원 점유 시간도 같다

### 짧아진다

생성이 시작되는 순간 화면에 글자가 나타난다  
사용자는 대기가 끝났다고 느낀다

스트리밍  
미적용

### 그대로이다

완료 시점은 스트리밍 여부와 무관하다  
비교의 기준이 된다

### 총 완료 시간과 같다

답변이 완성될 때까지 화면이 비어 있다  
체감 대기가 가장 길다

두 값을 함께 관리한다. TTFT만 보면 총 시간이 길어지는 변경을 놓치고, 총 시간만 보면 체감 개선을 놓친다.



# 단계별 지연 측정 코드

## 지연 측정

```
def timed_pipeline(q):
    t0 = time.perf_counter()
    ctx = retrieve(q)                # 질문 임베딩은 캐시 없이
    t1 = time.perf_counter()
    generate(q, ctx)
    t2 = time.perf_counter()
    return (t1 - t0), (t2 - t1)

r_lat, g_lat = [], []
for q in QUESTIONS[:30]:
    r, g = timed_pipeline(q)
    r_lat.append(r); g_lat.append(g)

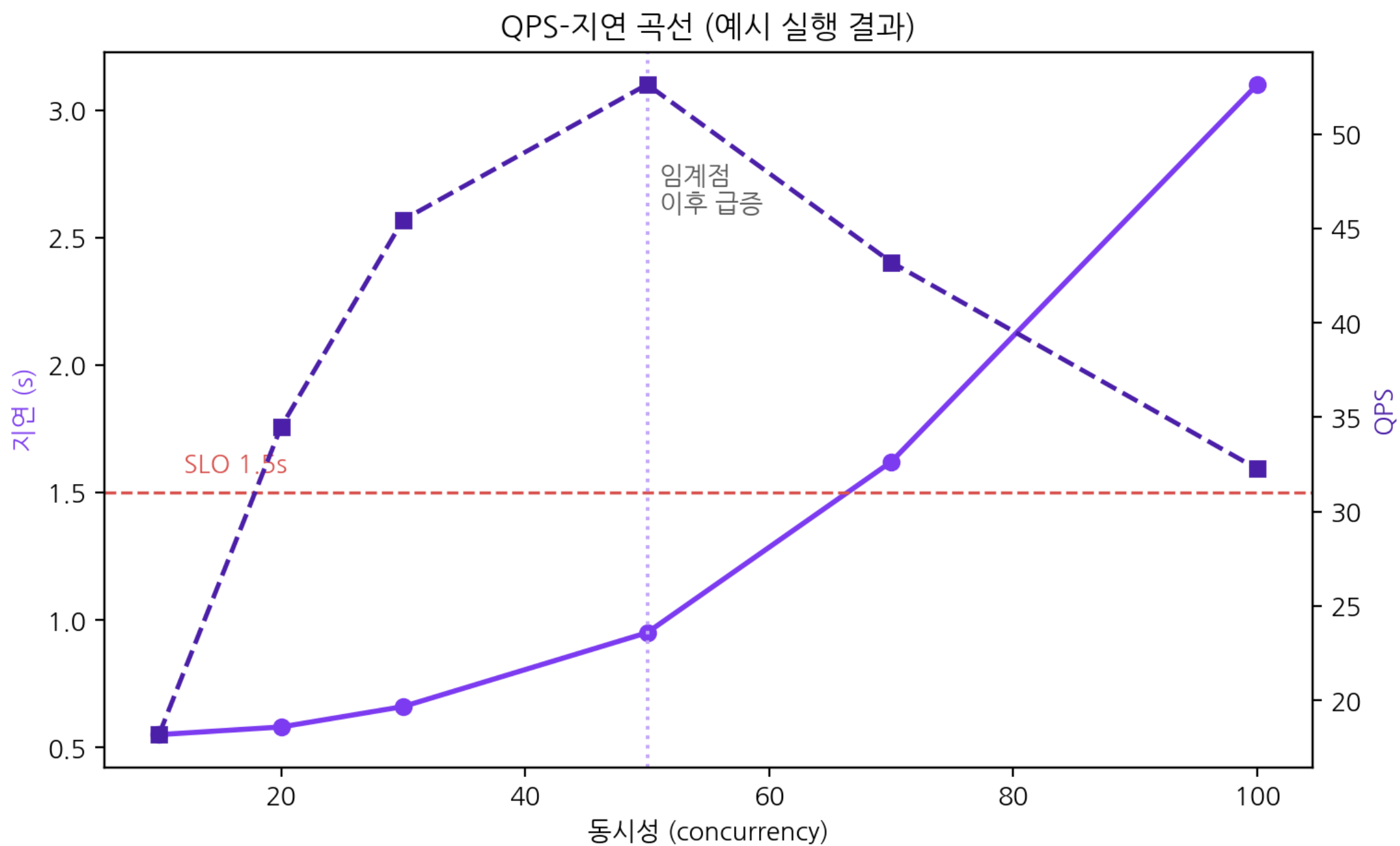
for name, xs in [("검색", r_lat),
                 ("생성", g_lat)]:
    print(name, pctl(xs, 50), pctl(xs, 95))
```

- perf\_counter로 검색과 생성을 분리 측정한다.
- 30회 반복해 p50·p95 분포로 본다. 1회 측정으로는 분포를 알 수 없다. 질문 임베딩에 캐시가 적용되면 검색 지연이 0에 가깝게 나오므로 캐시를 끄고 측정한다.
- 대개 생성이 병목이다. 비중을 숫자로 확인한다.

병목을 확정해야 개선 수단을 고를 수 있다. 검색 캐싱은 검색이 병목일 때만 의미가 있다.



# QPS-지연 곡선과 임계점



실습 11에서 생성하는 곡선 (예시 값)

## 임계점까지는 점진적으로 증가하고, 넘어서면 비선형으로 급증

동시성을 10→100으로 높이며 배치 지연과 QPS를 함께 관찰한 결과이다.

임계점 이전에는 동시성을 올려도 지연이 크게 늘지 않는다.  
임계점을 넘으면 지연이 비선형으로 급증하고 QPS는 더 늘지 않는다.  
SLO 선과 곡선이 만나는 지점이 운영 동시성 상한이다. 여유율은 30% 정도로 둔다.



# 지연 개선 수단 비교

| 수단        | 좋아지는 것      | 총 시간 단축 | 비용 영향 | 주의점              |
|-----------|-------------|---------|-------|------------------|
| 스트리밍      | TTFT(체감 대기) | 아니오     | 없음    | UI가 스트리밍을 지원해야 함 |
| 캐싱        | 반복 질문 응답 전체 | 예       | 절감    | 오래된 답 반환 위험      |
| 배치·비동기 처리 | 처리량(QPS)    | 경우에 따라  | 중립    | 지연·처리량 균형 설계 필요  |
| 모델 경량화    | 생성 지연·비용    | 예       | 절감    | 품질 재측정 필요        |

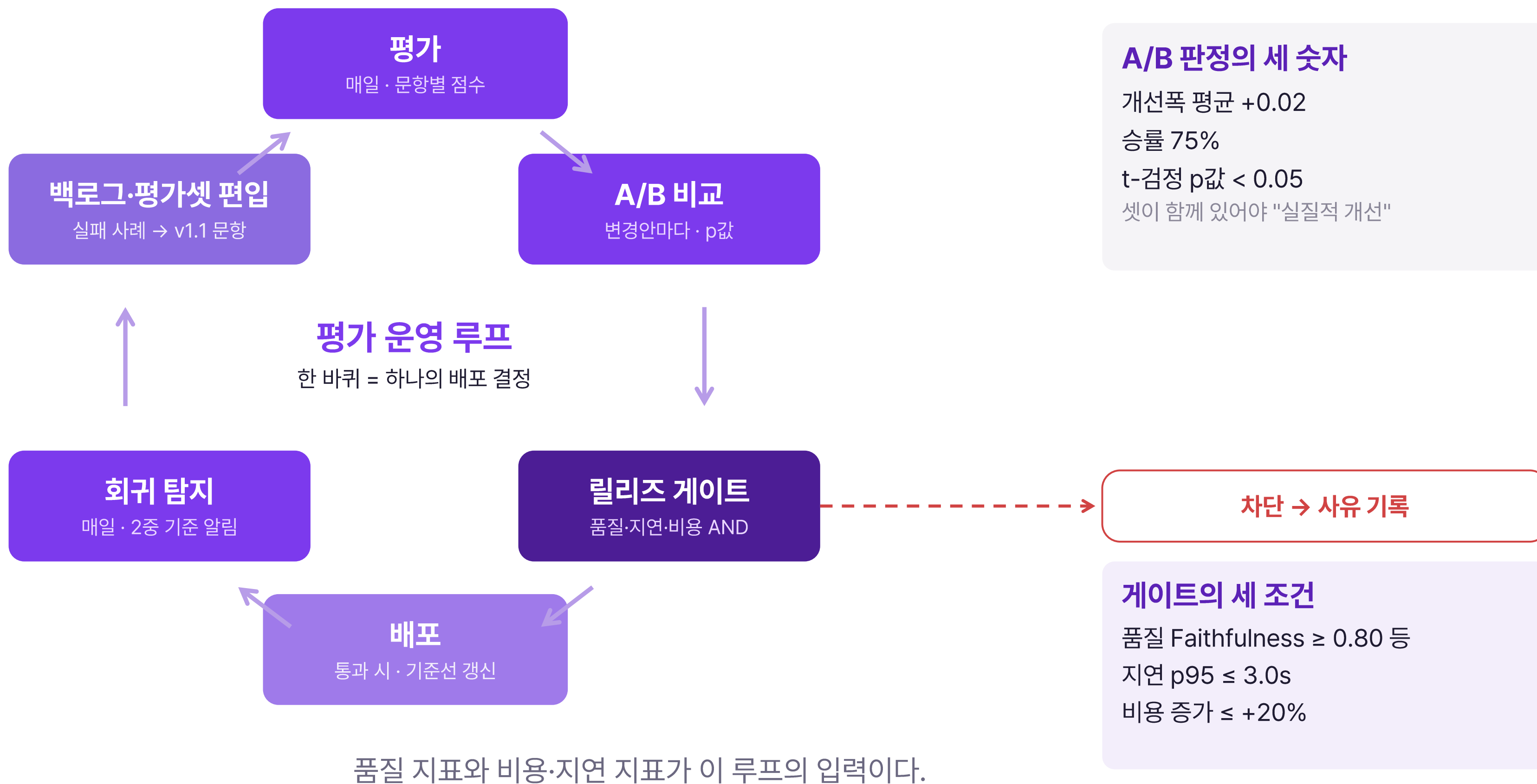
수단마다 좋아하는 지표가 다르다. TTFT·p95·QPS 중 무엇을 개선하려는지부터 정하고 수단을 고른다.

평가 운영 루프: A/B · 게이트 · 회귀



# 평가 운영 루프 전체 구조

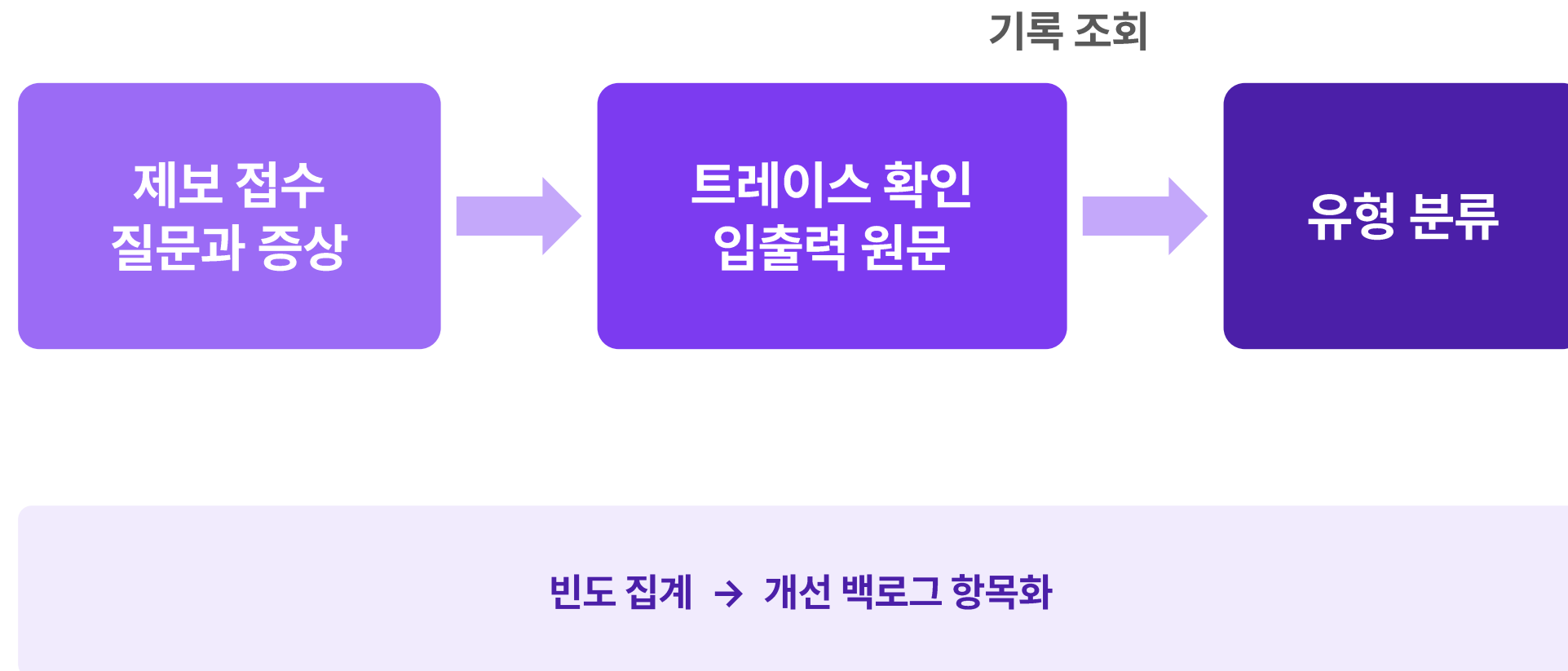
측정 결과를 배포 결정으로 연결하는 순환이다. 차단된 건도 백로그가 되어 루프를 다시 거친다.





# 실패 사례에서 개선 백로그까지

제보가 조치로 이어지려면 다섯 단계를 거친다. 기록이 없으면 2단계에서 끊긴다.



1. 제보 접수: 질문과 증상을 구체적으로 기록한다.
2. 트레이스 확인: 검색 청크·프롬프트·답변 원문을 본다.
3. 유형 분류: 검색·생성·컨텍스트 활용 실패 중 하나로 나눈다.
4. 빈도 집계: 유형별 건수로 우선순위를 정한다.
5. 백로그 항목화: 유형·건수·조치·확인할 지표를 남긴다.



# A/B 판정의 세 숫자

+0.02

평균 점수가 얼마나 올랐는가

개선폭 평균, 0.72→0.74  
수준의 차이는 이것만으로 판단 불가

75%

문항별로 얼마나 자주 우세했는가

승률, 문항별로 개선안이 우세한 비율. 절반 근처면 사실상  
무승부

$p < 0.05$

그 차이가 우연이 아닌가

같은 문항 짝을 비교하는 대응표본 t-검정의 p값

세 숫자가 함께 있어야 "실질적 개선"이다. 게이트에서 차단되면 사유를 기록해 개선 백로그로 잇는다.



# A/B 표본 크기와 최소 검출 효과

효과 크기 d는 평균 점수 차이를 문항별 차이의 표준편차로 나눈 값이다. d가 작을수록 필요한 문항이 빠르게 늘어난다.

| 검출하려는 차이 | 효과 크기 d | 필요한 문항 수 | 이 규모에서 가능한 것    | 주의                 |
|----------|---------|----------|-----------------|--------------------|
| 큰 차이     | 0.8     | 약 15문항   | 개선이 뚜렷할 때만 판별한다 | 작은 개선은 놓친다         |
| 중간 차이    | 0.5     | 약 34문항   | 실무에서 흔히 쓰는 규모이다 | 문항 구성이 결과에 크게 작용한다 |
| 작은 차이    | 0.3     | 약 90문항   | 미세 조정의 효과를 확인한다 | 채점 변동이 결과를 바꾼다     |
| 매우 작은 차이 | 0.2     | 약 199문항  | 대규모 평가셋이 필요하다   | 비용과 시간이 크게 늘어난다    |

표의 문항 수는 대응표본 t-검정 · 유의수준 0.05 · 검정력 0.80 기준이다.  
문항을 늘리기 어렵다면 판별할 수 있는 차이의 크기를 먼저 인정한다.



# t-검정과 릴리즈 게이트 코드

## A/B와 게이트

```
delta = float((impr - base).mean())
win_rate = float((impr > base).mean())
t, p = stats.ttest_rel(impr, base)

GATE = {
    "faithfulness_min": 0.80,
    "relevancy_min": 0.85,
    "recall_regress_margin": 0.02,
    "p95_max": 3.0,
    "cost_increase_max": 0.20,
}

def release_gate(m, baseline, gate=GATE):
    failed = []
    ... # 다섯 조건 검사
    return (not failed), failed
```

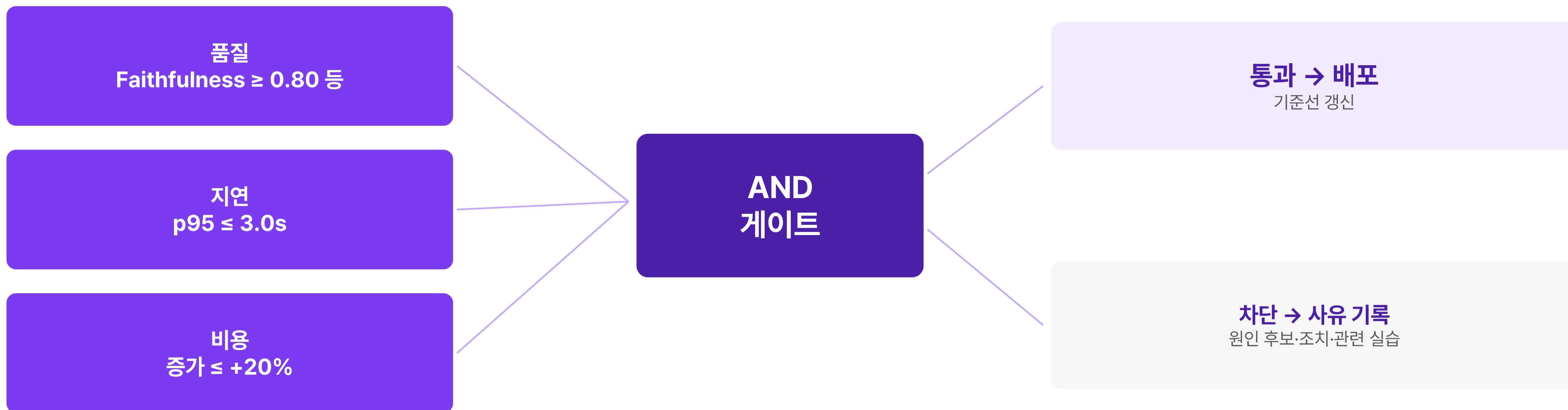
- ttest\_rel은 같은 문항 짝을 비교하는 대응표본 검정이다.
- 게이트는 절대 최저선과 상대 허용폭, 지연 상한을 함께 검사한다.
- 통과 여부와 함께 실패 사유 목록을 반환한다.

품질·지연·비용 세 기준의 임계값을 릴리즈 게이트에서 한 번에 검사한다.  
품질·비용·지연 지표가 모두 판정 근거가 된다.



# 릴리즈 게이트의 구조

세 조건을 모두 충족할 때만 통과하도록 구성한다. 하나라도 미달하면 사유와 함께 차단한다.



차단되었을 때 기준을 낮추는 것은 마지막 선택지이다. 개선안 수정을 먼저 검토하고, 회귀 허용폭에는 점수 변동성을 반영한다.



# 오프라인 A/B와 온라인 A/B

같은 변경을 배포 전에는 고정 평가셋으로, 배포 후에는 실사용 트래픽으로 확인한다.

## 오프라인 A/B · 배포 전

- 고정 평가셋으로 두 버전을 같은 조건에서 비교한다
- 사용자 노출이 없어 몇 번이든 다시 실행할 수 있다
- 평가셋이 실제 질문 분포와 어긋나면 결론도 어긋난다

판정 · 릴리즈 게이트 통과 여부

## 온라인 A/B · 배포 후

- 실사용 트래픽을 분할해 두 버전을 동시에 운영한다
- 재질문을 이탈처럼 평가셋에 없는 행동을 확인한다
- 품질이 낮은 버전이 사용자에게 노출되고, 유의성 확보까지 시간이 걸린다

판정 · 지표 악화 시 롤백

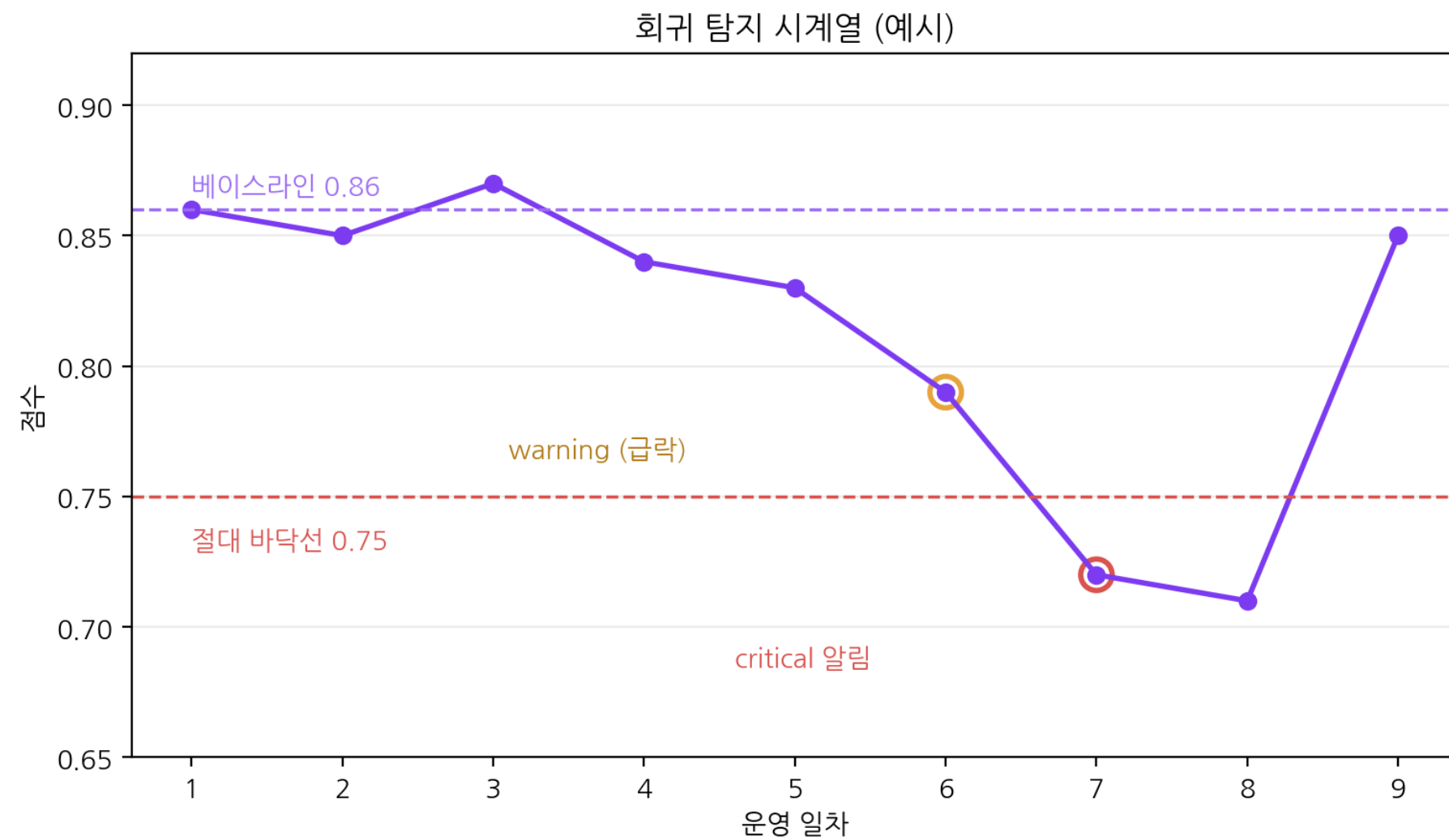
게이트가 동작하는 위치 · CI 파이프라인



수동으로 실행하는 게이트는 누락되기 쉽다. CI에 연결해 통과한 경우에만 병합되도록 구성한다.



# 회귀 탐지 시계열

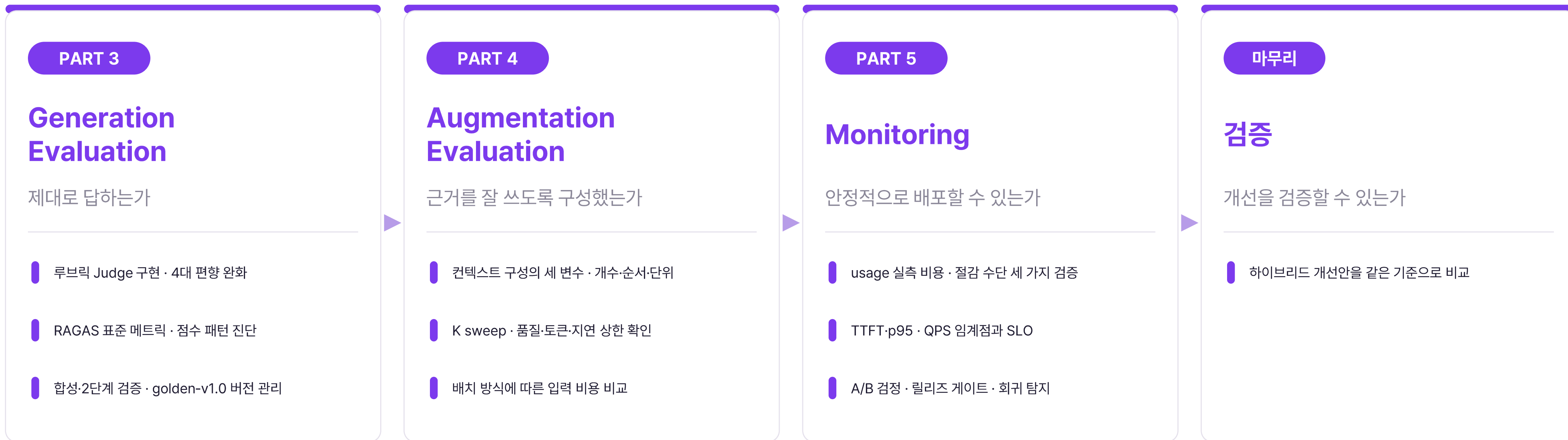


## 절대 최저선과 급락 감지의 이중 기준

매일 같은 평가셋을 실행해 점수 추이를 기록하면 드러나지 않던 품질 하락도 알림으로 감지할 수 있다.

절대 최저선(0.75) 미달은 심각도 critical로 즉시 알림이 발생한다.  
기준선 대비 급락(허용폭 초과)은 warning으로 분류한다.  
알림이 잦으면 임계값이 엄격한 것이고, 알림이 없으면 임계값이 관대하거나 품질이 안정적일 것이다.

"좋아졌다는 것을 어떻게 검증하는가?" 이제 보고서로 답할 수 있다.



이틀 동안 확보한 것

Golden Dataset v1.0

품질·지연·비용 세 가지 지표

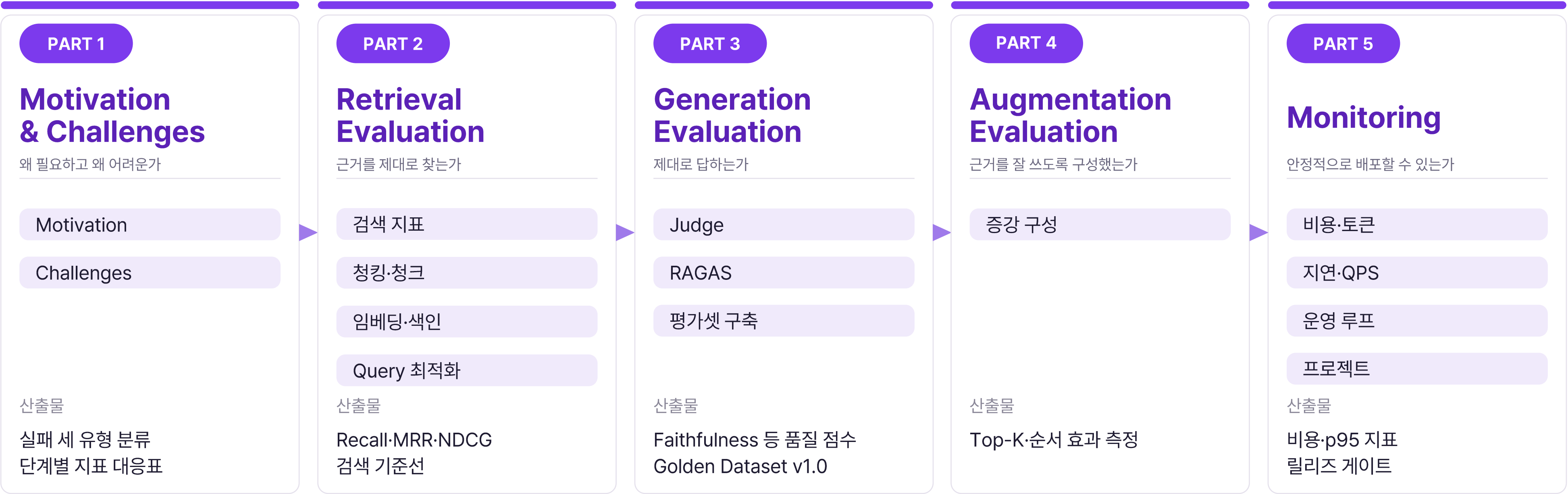
릴리즈 게이트와 운영 루프

성능 개선 보고서

측정에서 검증까지, 다섯 PART의 결과가 하나의 보고서로 정리된다.

숫자가 없으면 개선도 회귀도 확인할 수 없다. 이 과정에서 다룬 것은 그 숫자를 만드는 절차이다.

전체 과정은 평가의 필요성에서 시작해 개선 검증으로 끝난다.





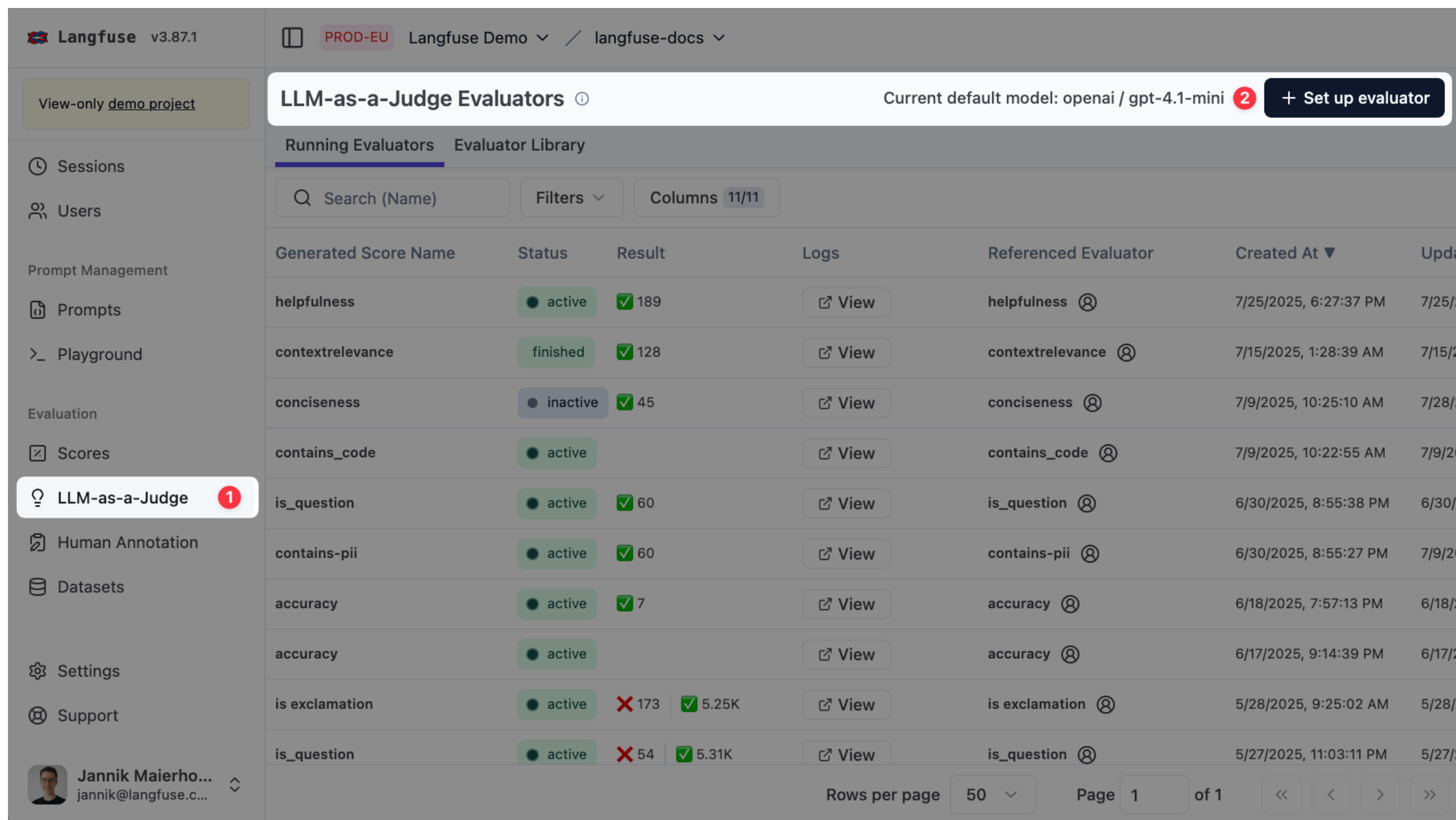
# 마무리 - 측정에서 검증까지

평가의 필요성에서 시작해 검증 보고서까지, 다섯 PART를 모두 다루었다.



이 운영 루프를 실제 서비스에 적용한다.  
검색 고도화(Semantic 청킹·재순위·Multi-Hop) 같은 개선안도 이 과정의 게이트로 판정한다.

실습에서 직접 구현한 Judge를, 운영에서는 LangFuse가 트레이스마다 자동으로 실행한다.



The screenshot shows the LangFuse v3.87.1 interface. The main section is titled 'LLM-as-a-Judge Evaluators' with a sub-header 'Running Evaluators'. It displays a table of evaluators with columns: Generated Score Name, Status, Result, Logs, Referenced Evaluator, Created At, and Updated At. The table lists several evaluators like 'helpfulness', 'contextrelevance', 'conciseness', 'contains\_code', 'is\_question', 'contains-pii', 'accuracy', 'is exclamation', and 'is\_question' with their respective statuses and results.

| Generated Score Name | Status   | Result | Logs                 | Referenced Evaluator | Created At             | Updated At             |
|----------------------|----------|--------|----------------------|----------------------|------------------------|------------------------|
| helpfulness          | active   | 189    | <a href="#">View</a> | helpfulness          | 7/25/2025, 6:27:37 PM  | 7/25/2025, 6:27:37 PM  |
| contextrelevance     | finished | 128    | <a href="#">View</a> | contextrelevance     | 7/15/2025, 1:28:39 AM  | 7/15/2025, 1:28:39 AM  |
| conciseness          | inactive | 45     | <a href="#">View</a> | conciseness          | 7/9/2025, 10:25:10 AM  | 7/28/2025, 10:25:10 AM |
| contains_code        | active   |        | <a href="#">View</a> | contains_code        | 7/9/2025, 10:22:55 AM  | 7/9/2025, 10:22:55 AM  |
| is_question          | active   | 60     | <a href="#">View</a> | is_question          | 6/30/2025, 8:55:38 PM  | 6/30/2025, 8:55:38 PM  |
| contains-pii         | active   | 60     | <a href="#">View</a> | contains-pii         | 6/30/2025, 8:55:27 PM  | 7/9/2025, 8:55:27 PM   |
| accuracy             | active   | 7      | <a href="#">View</a> | accuracy             | 6/18/2025, 7:57:13 PM  | 6/18/2025, 7:57:13 PM  |
| accuracy             | active   |        | <a href="#">View</a> | accuracy             | 6/17/2025, 9:14:39 PM  | 6/17/2025, 9:14:39 PM  |
| is exclamation       | active   | 173    | <a href="#">View</a> | is exclamation       | 5/28/2025, 9:25:02 AM  | 5/28/2025, 9:25:02 AM  |
| is_question          | active   | 54     | <a href="#">View</a> | is_question          | 5/27/2025, 11:03:11 PM | 5/27/2025, 11:03:11 PM |

## 운영 트래픽에 Judge를 상시 적용하는 화면

**1 설정**  
평가 프롬프트·척도·대상  
트레이스를 UI에서 설정한다

**2 실행**  
새 트레이스마다 Judge가 돌고  
점수가 트레이스에 스코어로 붙어 쌓인다

**3 활용**  
쌓인 점수 추이는 회귀 탐지의 입력이 된다

한 번 설정해 두면 운영 트래픽 전체가 상시 평가 대상이 된다.



한 번 측정된 지연은 그 시점의 값이다. 운영에서는 모든 호출의 지연이 자동으로 쌓여야 한다.

```
from langfuse.openai import openai
from langfuse import observe

# 임포트만 바꾸면 호출마다 지연 · 토큰 · 비용이 기록된다
client = openai.OpenAI()

@observe(name="retrieve")
def retrieve(q):          # 구간을 나눠 기록
    return vs.similarity_search(q, k=3)

@observe(name="rag")
def answer(q):
    ctx = retrieve(q)      # 검색 구간
    return client.chat.completions.create(
        model="gpt-5.4", messages=build(ctx, q))
```

## 1 자동 기록

래퍼를 거친 모든 호출의 지연이 트레이스에 기록된다. 계측 코드를 따로 추가하지 않는다.

## 2 구간 분해

검색과 생성을 각각 span으로 감싸면 어느 단계가 느린지 트레이스에서 바로 드러난다.

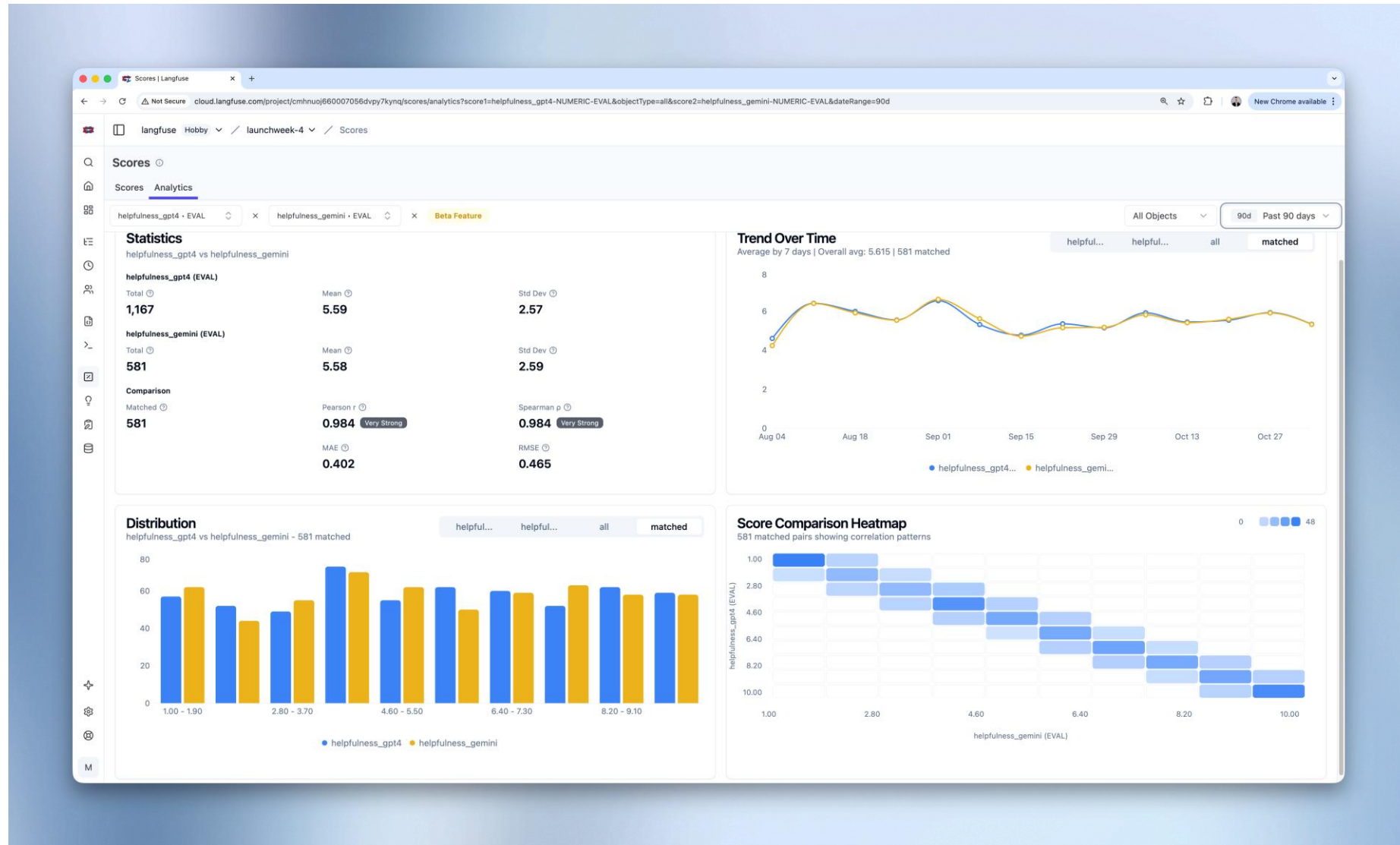
## 3 추이와 알림

대시보드에서 기간별 p50·p95를 보고, SLO를 초과하면 알림이 발생하도록 설정한다.

실습의 시뮬레이션은 구조를 익히기 위한 것이고, 운영에서의 분포는 이렇게 축적해 확인한다.



# LangFuse 스코어 대시보드



## 운영에서의 점수 추이 관찰

실습에서 직접 만든 시계열과 알림은 운영에서 대시보드와 알림 규칙으로 대체할 수 있다.

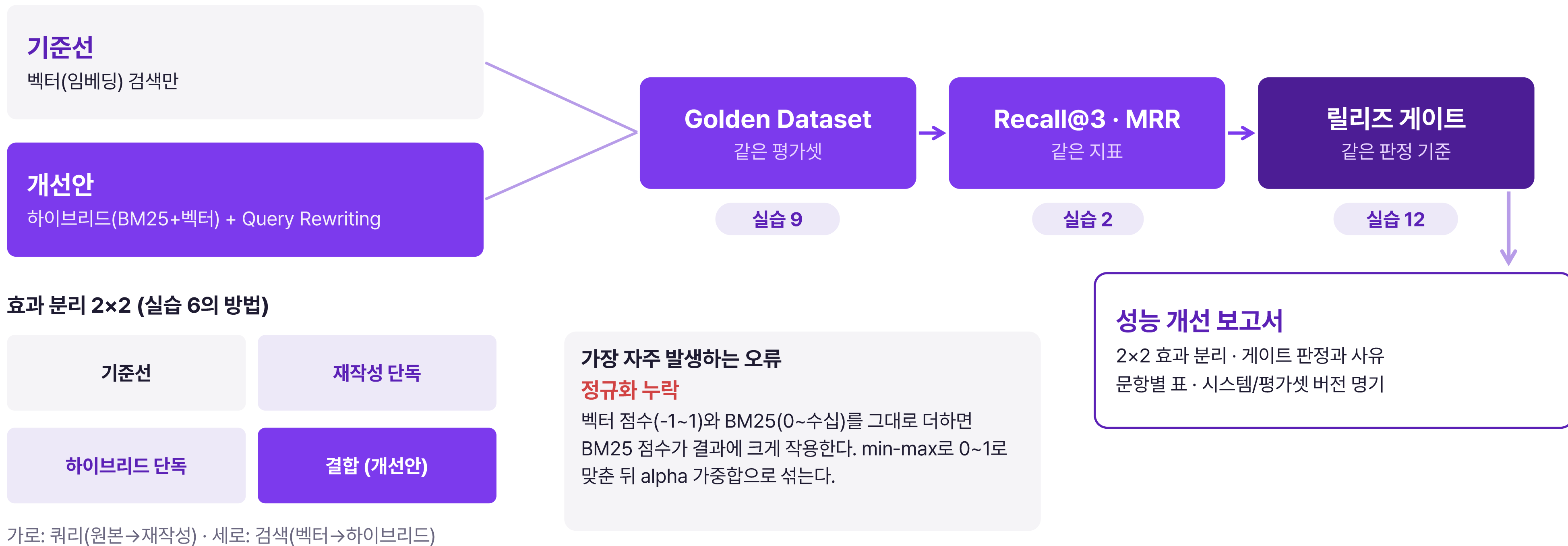
Judge 점수와 사용자 피드백이 스코어로 쌓인다.  
버전·모델·기간별 추이로 회귀 시점을 특정한다.  
임계값 알림을 설정하면 회귀 탐지가 자동으로 동작한다.

# Project: 하이브리드 RAG 성능 평가



# 프로젝트 아키텍처

목표는 개선 자체가 아니라 개선의 검증이다. 같은 평가셋, 같은 지표, 같은 게이트를 사용해야 검증이 성립한다.

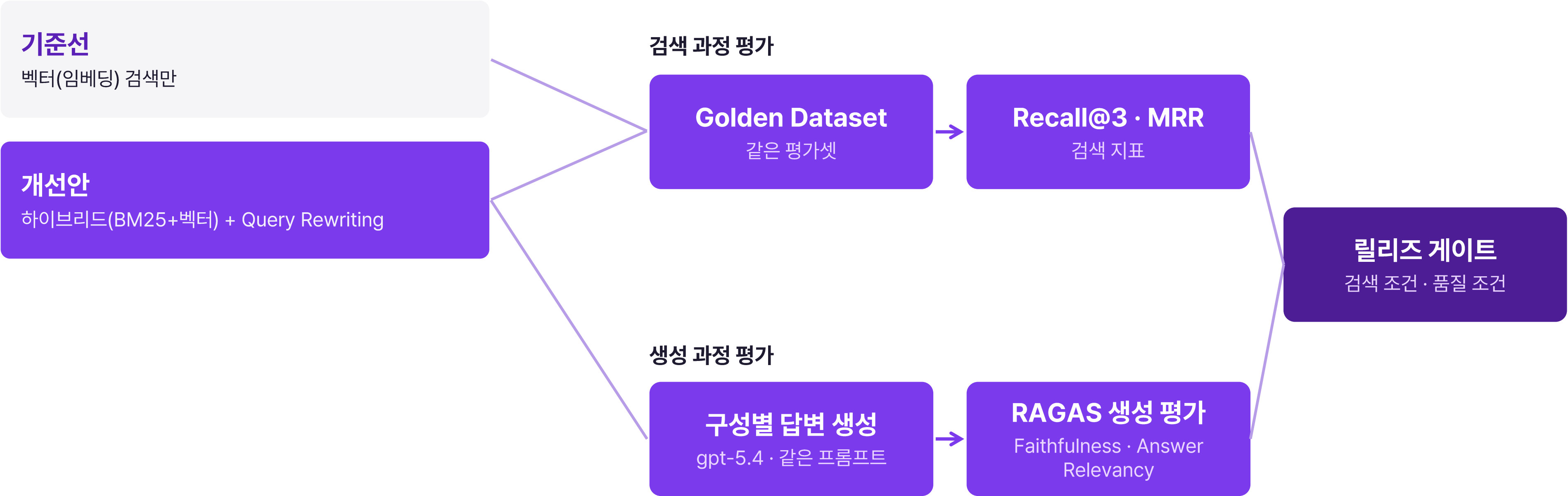


개선안이 기준선에 못 미쳐도 된다. 그 이유를 2×2와 문항별 표로 설명할 수 있으면 그것도 결과이다. 차단 사유는 다음 백로그가 된다.



# 생성 평가까지 포함한 평가 경로

검색 지표가 올라도 답변이 좋아졌다는 보장은 없다. 같은 구성으로 답변을 생성해 RAGAS로 함께 측정한다.



검색 지표만 오르고 Faithfulness가 떨어지면 개선으로 인정하지 않는다.



# 비용·지연이 게이트로 들어가는 흐름

요청별 실행 기록(TRACES)에서 비용과 지연을 추출해 게이트 조건으로 사용한다.

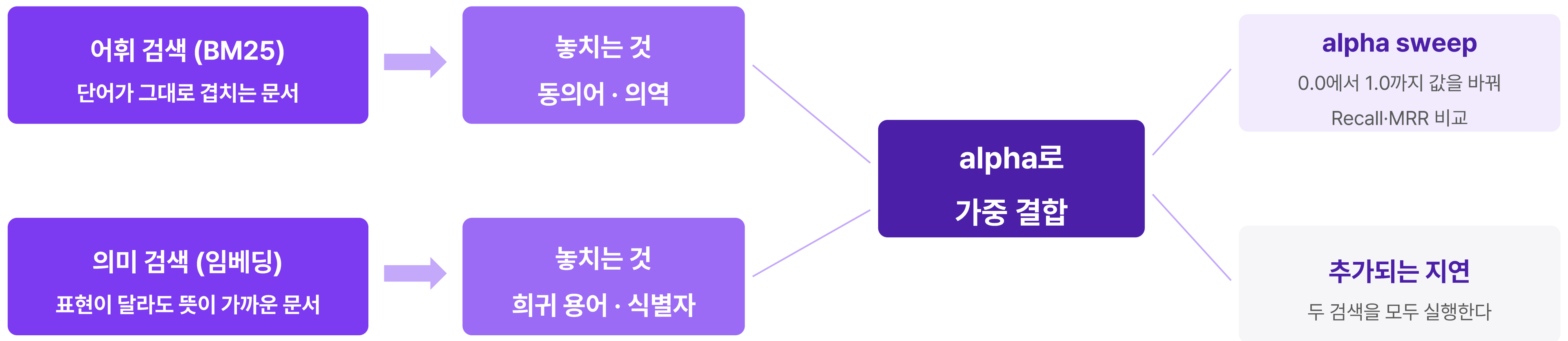


비용·지연은 품질과 같은 게이트에서 판정한다. 차단 사유는 그대로 개선 백로그의 다음 항목이 된다.



# 하이브리드 검색의 원리와 alpha

두 검색은 실패 유형이 서로 다르다. 하나의 점수로 섞으면서 비중을 alpha로 정한다.



alpha의 적정값은 코퍼스와 질문 유형에 따라 다르다. 같은 평가셋에서 값을 바꿔 가며 정한다.



# 하이브리드 랭킹과 정규화

## 하이브리드 랭킹

```
def _minmax(x):  
    x = np.asarray(x, dtype=float)  
    lo, hi = x.min(), x.max()  
    return ((x - lo) / (hi - lo)  
            if hi > lo else np.zeros_like(x))  
  
# 틀림 - 스케일이 달라 결과가 BM25 점수에 치우침  
score = vector_scores(q) + bm25_scores(q)  
  
# 맞음 - 각각 0~1로 맞춘 뒤 가중합  
score = (alpha * _minmax(vector_scores(q))  
         + (1 - alpha) * _minmax(bm25_scores(q)))
```

- 벡터 점수와 BM25는 스케일이 달라 그대로 더하면 안 된다.
- min-max로 0~1로 맞춘 뒤 alpha 가중합으로 섞는다.
- 한국어 BM25는 조사 문제로 음절 2-gram 토큰화를 사용한다.

자주 발생하는 오류가 정규화 누락이다. 정규화 여부에 따라 alpha 비교 결과가 달라지므로 같은 평가셋에서 확인한다.



# 효과 분리 2×2 실험

이번에는 검색 방식과 쿼리, 두 요인으로 네 구성을 비교한다.

|             | 원본 질문                                                                          | 재작성 질문                                                                      |
|-------------|--------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| 하이브리드<br>검색 | <div>하이브리드 단독 효과</div> <div>쿼리는 그대로 두고 검색 방식만 교체<br/>기준선과 비교해 검색 개선분만 측정</div> | <div>결합 효과 (개선안 전체)</div> <div>하이브리드 검색 + 질문 재작성<br/>제출할 개선안의 최종 구성</div>   |
| 벡터<br>검색만   | <div>기준선</div> <div>벡터 검색만 + 원본 질문<br/>네 구성 비교의 기준점</div>                      | <div>재작성 단독 효과</div> <div>검색은 그대로 두고 질문만 재작성<br/>기준선과 비교해 재작성 개선분만 측정</div> |

결합 효과가 단독 효과의 합보다 작을 수 있다. 따라서 네 구성을 모두 측정해 효과를 분리한다.



# 체크포인트와 판정 기준

| 체크포인트      | 확인 질문                 | 산출물                                  | 사용 도구                 |
|------------|-----------------------|--------------------------------------|-----------------------|
| CP1 기준선    | 개선 전 상태를 숫자로 남겼는가     | Recall@3 · MRR                       | 지표 함수                 |
| CP2 효과 분리  | 무엇이 효과를 냈는지 말할 수 있는가  | 2×2 표                                | 구성별 검색기               |
| CP3 게이트 판정 | 통과/차단과 사유를 설명할 수 있는가  | 판정 결과 · 사유                           | release_gate          |
| CP4 생성·유의성 | 답변 품질로 이어졌는가, 우연이 아닌가 | Faithfulness · Answer Relevancy · p값 | RAGAS evaluate · t-검정 |
| 최종 게이트     | 지연 조건까지 함께 판정했는가      | 생성 p95 ≤ 3.0s                        | release_gate_full     |
| 보고서        | 버전 두 개를 명기했는가         | 시스템·평가셋 버전                           | 구어체 평가셋 14문항          |

개선안이 기준선에 못 미치는 경우도 있다. 그 이유를 설명할 수 있으면 그것도 결과이다.